

Informe final de accesibilidad WCAG 2.2 AA - Cacao Leaf

Documento final generado en UTF-8 con fuente Unicode embebida para conservar correctamente tildes y caracteres del español.

Producto evaluado: Cacao Leaf

Tipo de producto: plataforma web y móvil para análisis preliminar de hojas de cacao mediante imágenes

Versión documentada: versión final desplegada para operación inicial

Fecha del informe: 17 de julio de 2026

Base normativa: Web Content Accessibility Guidelines (WCAG) 2.2, nivel AA

Alcance: experiencia web publicada, interfaz móvil compatible con Expo, API de diagnóstico, flujos de análisis, reportes e historial.

1. Resumen ejecutivo

Cacao Leaf es un producto digital orientado a productores, técnicos de campo y equipos agrícolas que necesitan registrar evidencias visuales de hojas de cacao, obtener una clasificación preliminar y dar seguimiento a casos observados. El sistema no reemplaza el diagnóstico profesional de un especialista agrícola; funciona como una herramienta de apoyo para priorizar revisión, registrar evidencia y mejorar progresivamente la calidad del análisis.

La revisión de accesibilidad se realizó sobre la versión final del producto, considerando las últimas actualizaciones implementadas: despliegue en nube, validación de imágenes que no corresponden, reporte de resultados incorrectos, reporte de rechazos, historial con trazabilidad, filtros por tipo de caso y comunicación clara de privacidad y mejora del sistema.

Este informe incorpora las últimas actualizaciones funcionales del producto y reemplaza versiones anteriores del documento de accesibilidad.

Conclusión general: no se identifican incumplimientos críticos desde la revisión de código y experiencia funcional. El producto presenta una base accesible y consistente con WCAG 2.2 AA para los flujos principales. Permanecen como validaciones recomendadas las pruebas manuales con lectores de pantalla, navegación por teclado en navegador, orientación horizontal y escalado de texto al 200%.

2. Estado final del producto

2.1 Despliegue y disponibilidad

- Frontend web: Netlify, sitio público de Cacao Leaf.
- Backend API: Render, servicio cacao-leaf-api.
- Repositorio: GitHub, rama principal main.
- Backend productivo: <https://cacao-leaf-api.onrender.com>.
- Configuración web: netlify.toml apunta el frontend al backend productivo mediante EXPO_PUBLIC_API_URL.

2.2 Funcionalidades principales evaluadas

- Captura de imagen desde cámara.
- Selección de imagen desde galería.

- Envío de imagen al backend mediante API REST.
- Validación de formato, tamaño y contenido visual de la imagen.
- Rechazo de imágenes claramente no relacionadas con hojas o vegetación.
- Clasificación preliminar como hoja aparentemente sana o posible patología visible.
- Resultado con nivel de confianza, observaciones y recomendación.
- Reporte de resultado cuando el usuario considera que la clasificación no coincide.
- Reporte de rechazo cuando el usuario considera que una imagen rechazada sí era válida.
- Historial con análisis, rechazos reportados, filtros y detalle de cada caso.
- Pantalla informativa con limitaciones, uso recomendado y aviso de privacidad/mejora.

2.3 Tecnologías del producto

- Frontend: React Native, Expo, TypeScript, React Native Web, lucide-react-native.
- Backend: Django, Django REST Framework, Gunicorn.
- Procesamiento de imagen: Pillow, NumPy, scikit-learn, h5py.
- Persistencia: SQLite en configuración base.
- Despliegue: Netlify para frontend web y Render para API.

3. Alcance de accesibilidad

La revisión cubre los flujos de usuario que forman parte de la experiencia final:

1. Inicio y acceso al análisis.
2. Captura o carga de imagen.
3. Estado sin imagen.
4. Estado de carga durante análisis.
5. Resultado preliminar.
6. Mensajes de error y rechazo de imagen.
7. Reporte de resultado.
8. Reporte de rechazo.
9. Historial con filtros: Todos, Reportados y Rechazos.
10. Detalle de análisis y detalle de rechazo.
11. Información de uso, limitación y privacidad.
12. Navegación inferior.

No se evalúa la precisión agronómica del clasificador como parte de WCAG. La accesibilidad se centra en percepción, operación, comprensión y compatibilidad técnica de la interfaz.

4. Criterios WCAG 2.2 AA considerados

Se revisaron los criterios más relevantes para el tipo de producto:

- 1.1.1 Contenido no textual: imágenes, iconos y miniaturas.
- 1.3.1 Información y relaciones: estructura de pantallas, encabezados, tarjetas y modales.
- 1.3.4 Orientación: uso en orientación vertical y horizontal.

- 1.4.1 Uso del color: estados que no dependen exclusivamente del color.
- 1.4.3 Contraste mínimo: contraste de textos principales y estados.
- 1.4.4 Cambio de tamaño del texto: compatibilidad con escalado del sistema.
- 2.1.1 Teclado: operación en web mediante controles enfocables.
- 2.1.2 Sin trampas de teclado: navegación esperada sin bloqueo de foco.
- 2.4.3 Orden del foco: lectura y navegación en orden lógico.
- 2.4.6 Encabezados y etiquetas: títulos y etiquetas descriptivas.
- 2.4.7 Foco visible: estados visuales de controles interactivos.
- 2.5.3 Etiqueta en el nombre: coherencia entre texto visible y nombre accesible.
- 2.5.8 Tamaño del objetivo: botones y controles táctiles de al menos 44 px.
- 3.2.1 Al recibir foco: ausencia de cambios inesperados por foco.
- 3.3.1 Identificación de errores: errores visibles y anunciados.
- 3.3.3 Sugerencias ante errores: mensajes que orientan la corrección.
- 4.1.2 Nombre, función, valor: roles, estados y nombres accesibles.
- 4.1.3 Mensajes de estado: carga, errores y confirmaciones.

5. Evaluación por experiencia de usuario

5.1 Inicio

La pantalla de inicio presenta el nombre del producto, una descripción breve y un botón principal para iniciar el análisis. La estructura es simple, sin sobrecarga visual, y utiliza pasos claros para explicar el uso.

Resultado: Cumple.

Evidencia: título principal, botón con etiqueta, pasos secuenciales y texto de apoyo.

5.2 Análisis de hoja

La pantalla de análisis permite cámara o galería, muestra el estado sin imagen, conserva una vista previa estable y comunica el procesamiento con un mensaje explícito. El mensaje de carga informa que el servicio puede tardar si estuvo inactivo, lo cual reduce incertidumbre en un despliegue con instancia gratuita.

Resultado: Cumple.

Evidencia: botones de acción claros, estado de carga, mensajes de error con `accessibilityRole="alert"`, imagen seleccionada con descripción accesible.

5.3 Resultado preliminar

El resultado muestra estado, confianza, observación y recomendación. No depende solo del color: usa texto, icono, porcentaje y contenido explicativo. Además, permite reportar un resultado si el usuario considera que no coincide.

Resultado: Cumple.

Evidencia: estado textual, porcentaje de confianza, notas, recomendación y acción de reporte.

5.4 Rechazo de imagen

Cuando la imagen no parece corresponder a una hoja o vegetación, el sistema muestra un mensaje accionable y permite reportar el rechazo. Esto evita que un posible falso rechazo quede sin trazabilidad.

Resultado: Cumple.

Evidencia: mensaje específico, acción Reportar rechazo, formulario con motivo y comentario opcional.

5.5 Historial y trazabilidad

El historial funciona como centro de revisión. Integra análisis normales, resultados reportados y rechazos reportados. Incluye filtros con contadores: Todos, Reportados y Rechazos. El detalle permite revisar evidencia, fecha, resultado, comentario y último reporte.

Resultado: Cumple.

Evidencia: filtros, contadores, etiquetas de estado, detalle modal, acción de reporte desde historial.

5.6 Información, limitaciones y privacidad

La pantalla informativa comunica patologías visibles, limitaciones del análisis, uso recomendado y uso de imágenes reportadas para revisión técnica y mejora del sistema.

Resultado: Cumple.

Evidencia: bloques temáticos con encabezados, texto claro y aviso de privacidad/mejora.

6. Resultados por criterio

Código	Criterio WCAG	Resultado	Evidencia principal	Riesgo residual
AX-01	1.1.1 Contenido no textual	Cumple	Imágenes y miniaturas tienen etiquetas; iconos decorativos se ocultan del lector	Validar lectura real con lector de pantalla
AX-02	1.3.1 Información y relaciones	Cumple	Encabezados, bloques, modales y filas siguen jerarquía lógica	Validación manual de orden de lectura
AX-03	1.3.4 Orientación	Cumple	Expo configurado con orientation: default	Revisar visualmente en landscape
AX-04	1.4.1 Uso del color	Cumple	Estados usan color, texto, iconos y etiquetas	Mantener si se agregan nuevos estados
AX-05	1.4.3 Contraste mínimo	Cumple	Paleta principal supera contraste AA en textos críticos	Recalcular si cambia la paleta
AX-06	1.4.4 Tamaño del texto	Cumple parcialmente	No se desactiva escalado de fuente	Probar texto al 200%
AX-07	2.1.1 Teclado	Cumple parcialmente	Controles interactivos basados en Pressable	Probar Tab, Enter, Espacio en web
AX-08	2.1.2 Sin trampas de teclado	Cumple parcialmente	No hay lógica que capture foco permanentemente	Probar modales en navegador
AX-09	2.4.3 Orden del foco	Cumple parcialmente	Orden visual y DOM siguen flujo lógico	Confirmar foco inicial y retorno
AX-10	2.4.6 Encabezados y etiquetas	Cumple	Títulos, botones, acciones y filtros son descriptivos	Mantener consistencia en futuras pantallas
AX-11	2.4.7 Foco visible	Cumple parcialmente	Estados visuales de controles presentes	Verificar foco en web con teclado
AX-12	2.5.3 Etiqueta en el nombre	Cumple	Texto visible coincide con nombres accesibles	Revisar nuevos botones futuros
AX-13	2.5.8 Tamaño del objetivo	Cumple	Botones principales y cierres cumplen mínimo táctil	Mantener mínimo de 44 px

Código	Criterio WCAG	Resultado	Evidencia principal	Riesgo residual
AX-14	3.3.1 Identificación de errores	Cumple	Errores visibles y anunciados	Validar anuncio con lector
AX-15	3.3.3 Sugerencias ante errores	Cumple	Mensajes indican formato, tamaño, contenido o permisos	Mantener mensajes específicos
AX-16	4.1.2 Nombre, función, valor	Cumple	Roles, estados, labels e hints en controles	Prueba manual con lector
AX-17	4.1.3 Mensajes de estado	Cumple	Carga, error y confirmación usan estados visibles	Validar live region en navegador/dispositivo

7. Hallazgos y mejoras aplicadas

7.1 Mejoras en comunicación de estado

Se reforzó el estado de carga durante el análisis con el mensaje: "Analizando imagen. Si el servicio estuvo inactivo, puede tardar unos segundos." Esto es relevante para una API desplegada en Render con instancia gratuita, donde el primer request puede demorar.

7.2 Mejora de trazabilidad operativa

El historial dejó de ser solo una lista de análisis. Ahora funciona como herramienta de revisión con filtros y contadores, diferenciando casos normales, reportados y rechazos.

7.3 Mejora de reportes

El producto permite reportar dos tipos de situaciones:

- Resultado generado que el usuario considera incorrecto.
- Imagen rechazada que el usuario considera válida.

Ambos casos se conservan como evidencia para revisión y mejora futura.

7.4 Mejora de textos y tono profesional

Se ajustó el tono de interfaz para producto final: etiquetas claras, advertencias explícitas y comunicación de limitaciones sin tratar la app como prototipo académico.

7.5 Privacidad y mejora

Se agregó un bloque informativo que comunica que las imágenes y observaciones reportadas pueden conservarse para revisión técnica y mejora del sistema.

8. Verificaciones técnicas realizadas

Durante el cierre del producto se ejecutaron verificaciones técnicas sobre frontend y backend:

- `python manage.py check`: correcto.
- `python manage.py test`: correcto, 12 pruebas automatizadas.
- `npm run build:web`: correcto, exportación web generada para Netlify.

Estas verificaciones no reemplazan una auditoría manual con tecnologías asistivas, pero reducen el riesgo de errores funcionales en la entrega.

9. Riesgos residuales

Los siguientes puntos quedan identificados como controles manuales recomendados antes de una certificación formal:

1. Probar TalkBack en Android.
2. Probar VoiceOver en iOS.
3. Verificar navegación por teclado en navegador: Tab, Shift+Tab, Enter y Espacio.
4. Confirmar foco inicial y retorno del foco en modales.
5. Probar escalado de texto al 200%.
6. Probar orientación horizontal en pantallas principales.
7. Ejecutar auditoría automatizada con Lighthouse o axe sobre la versión web publicada.

Ninguno de estos puntos representa una barrera crítica detectada desde el código; son validaciones de conformidad manual esperadas en un proceso formal WCAG.

10. Checklist final recomendado

Prueba	Resultado esperado
Abrir el sitio web publicado	La app carga sin errores y permite iniciar análisis
Usar cámara o galería	El usuario puede seleccionar una imagen
Analizar hoja válida	Se muestra resultado, confianza, observación y recomendación
Enviar imagen no relacionada	Se muestra rechazo con explicación y opción de reporte
Reportar resultado	El caso queda marcado como reportado
Reportar rechazo	El rechazo aparece en historial
Revisar historial	Se muestran filtros con contadores
Abrir detalle	El modal muestra datos del caso y permite cerrar
Consultar información	Se muestran limitaciones, uso recomendado y privacidad
Navegar con teclado en web	Los controles principales son alcanzables y operables
Usar lector de pantalla	Títulos, botones, estados y errores son anunciados correctamente

11. Dictamen final

Cacao Leaf presenta una experiencia alineada con WCAG 2.2 AA para los flujos principales de uso. La interfaz es operable, comprensible y consistente; los errores son visibles y accionables; los resultados no dependen exclusivamente del color; los controles principales tienen tamaño táctil adecuado; y el historial conserva trazabilidad de casos observados.

El producto se considera listo para entrega final y operación inicial, con la recomendación de ejecutar pruebas manuales con lectores de pantalla y teclado antes de declarar conformidad formal completa.

12. Archivos relacionados

- mobile/src/screens/HomeScreen.tsx
- mobile/src/screens/AnalyzeScreen.tsx
- mobile/src/screens/HistoryScreen.tsx
- mobile/src/screens/InfoScreen.tsx
- mobile/src/components/PrimaryButton.tsx
- mobile/src/components/TabBar.tsx
- mobile/src/theme/colors.ts
- mobile/src/api/client.ts
- backend/diagnostics/serializers.py
- backend/diagnostics/views.py
- netlify.toml
- render.yaml